

Prompt:

8/17/2026, 10:17:05 PM

lanjut Tahap AC

Response:

8/17/2026, 10:17:05 PM

Remembering

Reviewed Tahap AC, market microstructure, order flow, and QUBO trading workflows

Worked for 4s

Tahap AC — Market Microstructure & Order-Flow Engine

Baik. Kita lanjutkan secara konsisten dari Tahap AB.

Pada Tahap AB kita sudah membangun:

$$F_t \rightarrow \hat{\mu}_t, \hat{\Sigma}_t \rightarrow QUBO_t$$

Tetapi ada satu masalah fundamental:

Expected return belum tentu sama dengan expected executable return.

Misalnya model memperkirakan:

$$E[r_{t+1}] = 0.30\%.$$

Tetapi untuk mendapatkan posisi tersebut kita harus memasang limit order. Bisa saja:

- order tidak pernah filled,
- hanya terisi sebagian,
- harga bergerak sebelum fill,
- spread terlalu besar,
- market bergerak melawan kita,
- liquidity menghilang,
- atau setelah memperhitungkan fee + slippage, alpha menjadi negatif.

Maka Tahap AC akan menambahkan:

di antara forecast dan execution.

AC.1. Perubahan paradigma

Sebelumnya kita memiliki:

$$\mu_{i,t} = E[r_{i,t+1} | F_t].$$

Tetapi trader tidak mendapatkan:

$$r_{i,t+1}$$

secara gratis.

Kita membutuhkan execution decision:

$$a_{i,t} = (\text{side, price, quantity, order type}).$$

Maka quantity yang benar-benar terealisasi menjadi random variable:

$$Q_{i,t}^{fill}.$$

Sehingga return aktual dari sebuah order lebih tepat ditulis:

$$R_{i,t}^{exec} = Q_{i,t}^{fill} \cdot R_{i,t} - C_{i,t}.$$

AC.2. Order book sebagai keadaan pasar

Misalkan pada waktu t kita mempunyai order book:

$$OB_t = \{(b_k, q_k^b)_{k=1}^{K_b}, (a_k, q_k^a)_{k=1}^{K_a}\}.$$

dengan:

- b_k : bid price level ke- k ,
- q_k^b : bid quantity,

- a_k : ask price level ke- k ,
- q_k^a : ask quantity.

Best bid:

$$B_t = b_1.$$

Best ask:

$$A_t = a_1.$$

Mid-price:

$$M_t = \frac{A_t + B_t}{2}.$$

AC.3. Bid-ask spread

Spread absolut:

$$S_t = A_t - B_t.$$

Spread relatif:

$$s_t = \frac{A_t - B_t}{M_t}.$$

Ini penting karena sebuah strategi dapat mempunyai:

$$\hat{\mu} > 0$$

tetapi:

$$\hat{\mu} < s_t$$

sehingga tidak ekonomis untuk dieksekusi secara agresif.

AC.4. Microstructure-adjusted alpha

Maka expected alpha harus diubah menjadi:

$$\alpha_{i,t}^{net} = \mu_{i,t} - C_{i,t}^{fee} - C_{i,t}^{spread} - C_{i,t}^{slippage} - C_{i,t}^{funding}.$$

Untuk saham:

$$C^{funding}$$

biasanya tidak relevan seperti pada perpetual crypto.

Untuk crypto perpetual, funding dapat menjadi signifikan.

AC.5. Order Book Imbalance

Salah satu feature paling fundamental:

$$OBI_t = \frac{Q_t^b - Q_t^a}{Q_t^b + Q_t^a}.$$

dengan:

$$Q_t^b = \sum_{k=1}^K q_k^b$$

dan:

$$Q_t^a = \sum_{k=1}^K q_k^a.$$

Range:

$$-1 \leq OBI_t \leq 1.$$

Interpretasi:

$$OBI_t > 0$$

berarti depth bid lebih besar.

Sedangkan:

$$OBI_t < 0$$

berarti depth ask lebih besar.

AC.6. Tetapi depth level 1 saja terlalu sederhana

Kita dapat memberikan bobot terhadap setiap level:

$$w_k = e^{-\eta(k-1)}.$$

Kemudian:

$$Q_t^b = \sum_{k=1}^K w_k q_k^b,$$

dan:

$$Q_t^a = \sum_{k=1}^K w_k q_k^a.$$

Sehingga:

$$OBI_t^{(\eta)} = \frac{\sum_k w_k q_k^b - \sum_k w_k q_k^a}{\sum_k w_k q_k^b + \sum_k w_k q_k^a}.$$

Level terdekat mendapatkan bobot terbesar.

AC.7. Menghubungkan OBI dengan return

Sekarang kita dapat memperluas model AB:

$$r_{i,t+1} = \alpha_i + B_i f_t + \theta_i OBI_t + \epsilon_{i,t+1}.$$

Maka:

$$\hat{\mu}_{i,t+1} = \hat{\alpha}_i + \hat{B}_i f_t + \hat{\theta}_i OBI_t.$$

Tetapi kita harus hati-hati.

OBI bukan otomatis:

$$OBI > 0 \Rightarrow price \uparrow .$$

Hubungannya harus dipelajari dari data.

AC.8. Order Flow Imbalance

Order book adalah **state**.

Kita juga ingin mengetahui **perubahan state**.

Misalkan:

$$\Delta Q_t^b = Q_t^b - Q_{t-1}^b,$$

dan:

$$\Delta Q_t^a = Q_t^a - Q_{t-1}^a.$$

Maka sebuah bentuk sederhana:

$$\boxed{OFI_t = \Delta Q_t^b - \Delta Q_t^a.}$$

OFI menangkap tekanan order flow.

AC.9. Trade imbalance

Jika tersedia data transaksi:

$$V_t^{buy}$$

dan:

$$V_t^{sell},$$

kita dapat mendefinisikan:

$$TI_t = \frac{V_t^{buy} - V_t^{sell}}{V_t^{buy} + V_t^{sell}}.$$

Ini berbeda dari OBI.

OBI:

resting liquidity.

TI:

executed liquidity.

AC.10. Menggabungkan tiga dimensi

Sekarang kita mempunyai:

$$X_t^{micro} = [OBI_t, OFI_t, TI_t, Spread_t, Depth_t, \dots].$$

Kemudian:

$$\mu_{i,t} = f(X_t^{macro}, X_t^{technical}, X_t^{factor}, X_t^{micro}).$$

Namun saya tetap ingin model baseline dibuat sederhana terlebih dahulu.

AC.11. Market impact

Sekarang kita masuk ke bagian yang sangat penting.

Misalkan kita ingin membeli:

$$Q$$

unit.

Jika:

$$Q \leq q_1^a,$$

kita mungkin dapat membeli semuanya pada:

$$A_t.$$

Tetapi jika:

$$Q > q_1^a,$$

kita harus memakan level berikutnya:

$$a_2, a_3, \dots$$

AC.12. VWAP execution price

Jika quantity yang dieksekusi pada level k adalah:

$$q_k^{exec},$$

maka:

$$P^{exec} = \frac{\sum_k p_k q_k^{exec}}{\sum_k q_k^{exec}}.$$

Slippage terhadap mid:

$$Slip = P^{exec} - M_t$$

untuk buy.

Untuk sell:

$$Slip = M_t - P^{exec}.$$

AC.13. Market impact function

Untuk ukuran order yang lebih besar, kita dapat memodelkan:

$$I(Q) = \eta \left(\frac{Q}{ADV} \right)^\alpha.$$

dengan:

- Q : order size,
- ADV : average daily volume,
- η : impact scale,
- α : impact exponent.

Sering kali:

$$0 < \alpha < 1.$$

Jadi market impact tidak selalu linear terhadap ukuran order.

AC.14. Implementation shortfall

Jika kita memiliki target price:

$$P^{decision},$$

dan execution price:

$$P^{exec},$$

maka:

$$IS = P^{exec} - P^{decision}$$

untuk long.

Secara lengkap:

$$IS = Spread + Slippage + MarketImpact + Fees.$$

AC.15. Sekarang masuk ke inti strategi kita: limit order

Kita tidak selalu ingin langsung membeli.

Misalnya:

$$M_t = 100.$$

Kita memperkirakan:

$$P_{future} = 102.$$

Kita dapat:

Market order

beli sekarang di:

100.05.

atau:

Limit order

letakkan:

99.80.

Masalahnya:

99.80

mungkin tidak pernah tersentuh.

AC.16. Maka fill probability menjadi variabel utama

Definisikan:

$$p^{fill} = P(Q^{fill} \geq Q \mid OB_t, F_t, a_t).$$

Ini adalah probabilitas order terisi.

AC.17. Expected executable alpha

Misalkan:

- profit jika filled = G ,
- loss/opportunity cost jika tidak filled = L .

Maka:

$$E[\Pi] = p^{fill}G - (1 - p^{fill})L.$$

Namun kita dapat membuatnya lebih konkret.

AC.18. Limit order expected value

Misalkan limit buy pada harga:

$$P_L.$$

Jika filled, expected future price:

$$E[P_{future} | fill].$$

Profit per unit:

$$G = E[P_{future} | fill] - P_L.$$

Jika order tidak filled, kita dapat mengalami opportunity cost:

$$L = E[P_{future} | no fill] - P_L.$$

Maka:

$$EV_{limit} = p_{fill}G - (1 - p_{fill})L - C.$$

AC.19. Ini jauh lebih baik daripada sekadar signal

Sistem sebelumnya mungkin berkata:

$$\hat{\mu} > 0 \Rightarrow BUY.$$

Sekarang:

$$\hat{\mu} > 0 \Leftrightarrow BUY.$$

Kita memerlukan:

$$EV_{\text{execution}} > 0.$$

AC.20. Market order versus limit order

Kita dapat menghitung:

$$EV_{\text{market}}$$

dan:

$$EV_{\text{limit}}.$$

Kemudian:

$$a_t^* = \arg \max_{a \in \{\text{market}, \text{limit}, \text{pass}\}} EV(a).$$

Ini merupakan **execution optimization problem**.

AC.21. Bahkan dapat diperluas

Action space:

$$a_t = (\text{side}, \text{price}, \text{quantity}, \text{type}, \text{TTL}).$$

Contoh:

$$a_t = (\text{BUY}, 99.80, 0.5, \text{LIMIT}, 30\text{min}).$$

Jadi kita tidak hanya mengoptimalkan:

asset selection.

Kita juga mengoptimalkan:

how to enter.

AC.22. Time-to-live

Misalkan limit order memiliki TTL:

$$\tau.$$

Kita definisikan:

$$p_{fill}(\tau).$$

Biasanya:

$$p_{fill}(\tau_2) > p_{fill}(\tau_1)$$

jika:

$$\tau_2 > \tau_1,$$

karena order memiliki lebih banyak waktu untuk terkena.

Tetapi opportunity cost juga meningkat.

AC.23. Fill probability sederhana

Sebagai baseline:

$$p_{fill} = \sigma(\theta_0 + \theta_1 d + \theta_2 OBI + \theta_3 OFI + \theta_4 \tau + \theta_5 Q_{ahead})$$

dengan sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

Di sini:

$$d = \frac{|P_L - M_t|}{M_t}.$$

AC.24. Queue position

Ini sangat penting.

Misalkan kita memasang limit buy pada:

$$P_L.$$

Di depan kita terdapat:

$$Q_{ahead}.$$

Jika:

$$Q_{ahead} = 1000$$

unit, maka order kita tidak akan filled hanya karena harga menyentuh level tersebut.

Kita juga membutuhkan:

queue depletion.

AC.25. Queue depletion model

Misalkan market sell volume yang masuk:

$$V_t^{sell}.$$

Queue position setelah periode τ :

$$Q_{ahead}(\tau) = Q_{ahead}(0) - D(\tau),$$

dengan:

$$D(\tau)$$

adalah cumulative queue depletion.

Order filled ketika:

$D(\tau) \geq Q_{ahead}(0).$

AC.26. Probabilistic queue model

Karena kita tidak tahu future order flow secara pasti:

$$D(\tau)$$

adalah random variable.

Maka:

$$p_{fill}(\tau) = P(D(\tau) \geq Q_{ahead} \mid F_t).$$

Ini secara matematis jauh lebih masuk akal daripada:

"Harga menyentuh limit berarti order filled."

AC.27. Partial fill

Misalkan:

$$Q = 100.$$

Tetapi hanya:

$$60$$

unit yang filled.

Maka:

$$Q^{fill} = 60.$$

Kita dapat mendefinisikan fill ratio:

$$FR = \frac{Q^{fill}}{Q}.$$

dengan:

$$0 \leq FR \leq 1.$$

AC.28. Expected fill quantity

Daripada hanya:

$$p_{fill},$$

kita dapat menghitung:

$$E[Q^{fill}] = Q \cdot E[FR].$$

Ini lebih tepat untuk position sizing.

AC.29. Execution-adjusted expected return

Sekarang kita gabungkan semuanya.

Misalkan:

$$\hat{\mu}_i$$

adalah expected price return.

Kemudian:

$$p_{fill}$$

adalah fill probability.

Expected executable alpha:

$$\hat{\alpha}_i^{exec} = p_{fill} \hat{\mu}_i - C_{exec}.$$

Untuk partial fill:

$$\hat{\alpha}_i^{exec} = E[FR] \hat{\mu}_i - C_{exec}.$$

AC.30. Tetapi ini masih belum cukup

Karena:

$$P_{fill}$$

dan:

$$\hat{\mu}$$

tidak independent.

Misalnya kita memasang limit buy jauh di bawah market.

Jika market turun sampai limit tersebut:

$$P_{fill} \uparrow .$$

Tetapi kondisi tersebut mungkin berarti:

$$E[P_{future} \mid fill]$$

juga berubah.

Jadi:

$$\boxed{P(fill)}$$

dan:

$$P(return \mid fill)$$

harus dimodelkan bersama.

AC.31. Selection effect

Secara probabilistik:

$$E[R] = P(F)E[R \mid F] + P(F^c)E[R \mid F^c].$$

Ini sangat penting.

Karena order yang filled bukan sampel random.

$\boxed{\text{Fill}}$

sendiri membawa informasi tentang market path.}

AC.32. Contoh intuitif

Misalkan kita memasang:

$BUY\ 99.8$

ketika market:

$100.$

Jika harga langsung naik:

$100 \rightarrow 101,$

order tidak filled.

Jika harga:

$100 \rightarrow 99.8,$

order filled.

Tetapi kondisi kedua berarti market terlebih dahulu bergerak turun.

Jadi:

$P(\text{future return} \mid \text{fill})$

bisa berbeda jauh dari:

$P(\text{future return}).$

AC.33. Ini menghasilkan konsep penting

Kita membutuhkan:

$\boxed{\text{Conditional Alpha.}}$

Yaitu:

$$\alpha^{fill} = E[R_{future} | F_{ill}, F_t].$$

Bukan hanya:

$$E[R_{future} | F_t].$$

AC.34. Execution-aware alpha

Maka:

$$\alpha^{exec} = p_{fill} (E[R | F_{ill}] - C).$$

Tetapi kita juga dapat memasukkan opportunity cost:

$$EV_{limit} = p_{fill} \alpha^{fill} - (1 - p_{fill}) \alpha^{miss} - C.$$

AC.35. Ini mengubah QUBO kita

Sebelumnya:

$$\min_x [-\mu^T x + \lambda x^T \Sigma x].$$

Sekarang:

$$\min_x [-(\alpha^{exec})^T x + \lambda x^T \Sigma x].$$

Dengan:

$$\alpha^{exec} = f(\mu, OBI, OFI, Spread, Depth, p_{fill}, Slippage, Fees).$$

Jadi QUBO sekarang memilih asset berdasarkan **expected executable alpha**, bukan theoretical alpha.

AC.36. Order price juga dapat menjadi optimization variable

Ini bahkan lebih menarik.

Misalkan tersedia kandidat price:

$$p_1, p_2, \dots, p_K.$$

Untuk setiap:

$$p_k$$

kita hitung:

$$EV_{i,k}.$$

Kemudian:

$$p_i^* = \arg \max_{p_k} EV_{i,k}.$$

Jadi kita tidak lagi mengatakan:

"Beli SOL."

Tetapi:

"Jika membeli SOL, harga limit optimal adalah p^* ."

AC.37. Discretization

Kita dapat membentuk:

$$p_k = M_t - k\Delta p.$$

Misalnya:

$$k = 0, 1, \dots, 10.$$

Kemudian setiap candidate memiliki:

$$p_{fill,k},$$

$$slippage_k,$$

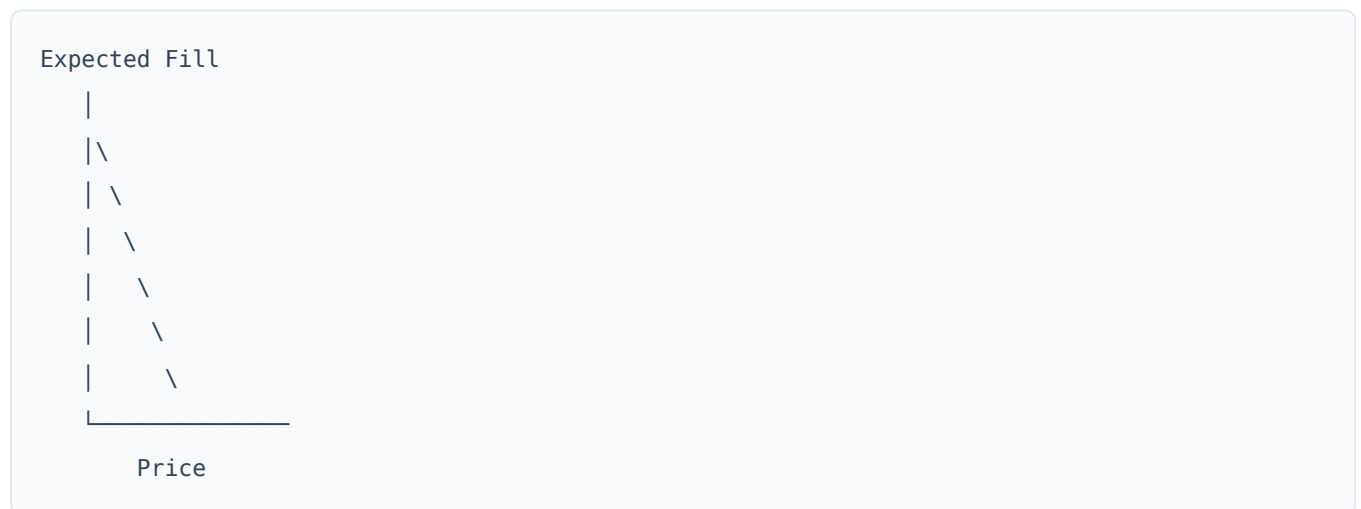
$$EV_k.$$

AC.38. Execution frontier

Kita akan mendapatkan trade-off:

Better Price $\leftrightarrow$ Lower Fill Probability.

Secara konseptual:



Semakin agresif order:

$$p_{fill} \uparrow$$

tetapi:

$$P^{exec} \uparrow$$

untuk buy.

Semakin pasif:

$$P^{exec} \downarrow$$

tetapi:

$$p_{fill} \downarrow .$$

AC.39. Optimal execution

Maka:

$$a^* = \arg \max_a EV(a).$$

Ini sudah merupakan optimization problem tersendiri.

Dan menariknya, problem ini juga dapat diformulasikan sebagai:

$$QUBO$$

jika kita ingin menggunakan pendekatan quantum untuk pemilihan discrete execution actions.

AC.40. Contoh QUBO execution

Misalkan kita memiliki tiga kandidat:

$$a_1 = \text{market},$$

$$a_2 = \text{limit} - 1\text{tick},$$

$$a_3 = \text{limit} - 2\text{tick}.$$

Definisikan:

$$x_k \in \{0, 1\}.$$

Constraint:

$$x_1 + x_2 + x_3 = 1.$$

Objective:

$$\min_x - \sum_{k=1}^3 EV_k x_k + P \left(\sum_k x_k - 1 \right)^2 .$$

Ini sudah menjadi QUBO.

AC.41. Tetapi saya tidak ingin langsung melakukan ini

Karena kita sekarang mempunyai **dua optimization layer**:

Layer 1 — Portfolio optimization

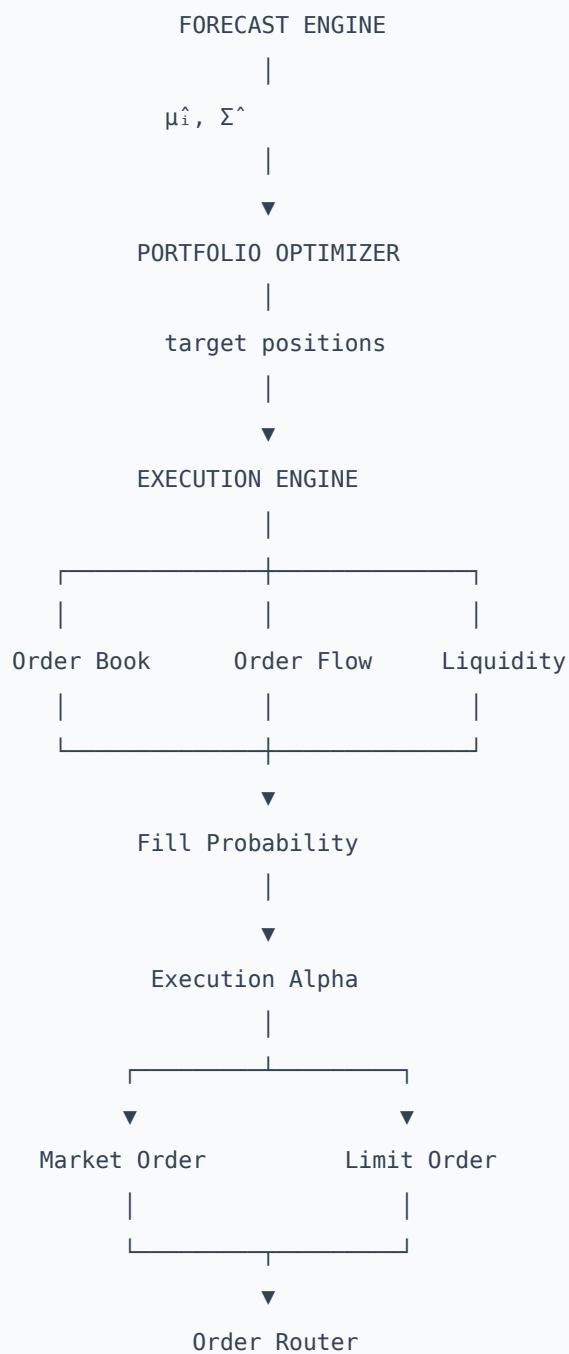
Which assets?

Layer 2 — Execution optimization

How to execute them?

Sebaiknya kita pisahkan dahulu.

AC.42. Arsitektur AC yang saya sarankan



AC.43. Ini cocok dengan desain awal kita

Karena sebelumnya kita ingin:

data → menentukan order book → TP/SL → evaluasi periodik.

Sekarang bagian "menentukan order book" mempunyai dasar matematis:

$$\boxed{\text{Order Placement} = \text{Optimization}}$$

bukan sekadar rule:

"pasang limit 0.5% di bawah harga."

AC.44. Stop Loss juga masuk ke microstructure

Misalkan entry:

$$P_E.$$

Stop:

$$P_{SL}.$$

Gross risk:

$$R_{gross} = |P_E - P_{SL}|Q.$$

Tetapi actual loss:

$$\boxed{R_{actual} = R_{gross} + \text{slippage}_{SL} + \text{fee}_{SL}.$$

Jadi target:

$$R = \$1$$

harus menggunakan:

$$\boxed{R_{actual} \leq 1.}$$

Bukan hanya:

$$R_{gross} = 1.$$

AC.45. Stop-loss slippage

Ini sangat penting pada kondisi cepat.

Jika:

$$P_{SL} = 99$$

tetapi market bergerak:

$$99.2 \rightarrow 98.5,$$

stop execution mungkin terjadi di:

$$98.7$$

atau lebih buruk.

Maka:

$$P_{SL}^{theoretical} \neq P_{SL}^{executed}.$$

Kita perlu:

$$E[Loss_{SL}]$$

dan:

$$Var_{SL}$$

untuk sizing.

AC.46. Expected stop loss

Misalkan slippage stop:

$$S_{SL}$$

adalah random variable.

Maka:

$$E[L] = Q[P_E - E[P_{SL}^{exec}]].$$

Target risk:

$$E[L] \leq \$1$$

untuk baseline.

Tetapi untuk risk control yang lebih konservatif, kita nantinya dapat menggunakan quantile:

$$VaR_{\alpha}(L) \leq \$1.$$

AC.47. Ini akan sangat berguna untuk crypto

Karena pada market yang volatile:

$$E[slippage]$$

dapat berubah cepat.

Maka:

$$Q_t$$

harus dinamis.

Misalnya:

$$Q_t = \frac{1}{d_{SL} + E[slippage_{SL}] + fee}.$$

AC.48. Take Profit juga bukan sekadar fixed percentage

Misalkan:

$$TP = +2R.$$

Itu hanya static rule.

Kita dapat memperkirakan:

$$P(P_{future} \geq TP \mid F_t).$$

Definisikan:

$$p_{TP} = P(\text{hit } TP \text{ before } SL \mid F_t).$$

AC.49. Expected value TP/SL

Misalkan:

$$G = TP - P_E$$

dan:

$$L = P_E - SL.$$

Maka:

$$EV = p_{TP}G - (1 - p_{TP})L - C.$$

Trade hanya layak jika:

$$EV > 0.$$

AC.50. Ini membawa kita ke konsep path probability

Bukan hanya:

$$P(P_{t+H} > TP).$$

Kita memerlukan:

$$P(\tau_{TP} < \tau_{SL} \mid F_t)$$

dengan:

$$\tau_{TP} = \inf \{s : P_s \geq TP\}$$

dan:

$$\tau_{SL} = \inf \{s : P_s \leq SL\}.$$

Ini adalah **first-passage probability**.

Dan ini jauh lebih sesuai untuk sistem trading kita.

AC.51. Sekarang hubungan dengan tahap berikutnya mulai terlihat

Kita nantinya dapat menghitung:

$$P(TP \text{ before } SL)$$

menggunakan:

- historical path,
- stochastic process,
- empirical distribution,
- regime,
- volatility,
- order flow.

Kemudian:

$$TP$$

dan:

$$SL$$

tidak lagi harus fixed.

AC.52. Dynamic TP

Misalkan posisi sedang profit:

$$P_t > P_E.$$

Kita dapat menghitung ulang:

$$EV(TP_k, SL_k)$$

untuk berbagai candidate:

$$TP_k.$$

Kemudian:

$$TP_t^* = \arg \max_{TP_k} EV(TP_k, SL_t).$$

Inilah fondasi matematis dari ide yang kamu sampaikan sebelumnya:

jika floating profit tinggi, apakah TP layak dinaikkan?

Jawabannya nanti:

$$\text{naikkan } TP \text{ jika } EV(TP_{new}) > EV(TP_{current}).$$

AC.53. Dynamic Stop Loss

Demikian pula:

$$SL_t^* = \arg \max_{SL_k} EV(TP, SL_k)$$

dengan constraint:

$$Risk(SL_k) \leq R_{max}.$$

Ini dapat menghasilkan:

$$SL : \text{initial} \rightarrow BEP \rightarrow \text{trailing} \rightarrow \text{profit} - \text{lock}.$$

Tetapi **bukan berdasarkan fixed percentage**.

Melainkan:

$$\text{expected value} + \text{risk} + \text{market state}.$$

AC.54. Tahap AC akhirnya menghasilkan empat output

Market microstructure engine kita akan menghasilkan:

$$M_t = \{p_{fill}, E[P^{exec}], E[slippage], E[impact]\}.$$

Kemudian:

$$\alpha_t^{exec} = g(\hat{\mu}_t, M_t, fees, funding).$$

Dan:

$$Risk_t^{exec} = h(\hat{\Sigma}_t, slippage, liquidity).$$

AC.55. Pipeline sekarang menjadi jauh lebih lengkap

MarketData



Feature&FactorEngine



$\hat{\mu}_t, \hat{\Sigma}_t$



Portfolio Optimization



Target Position

↓

Microstructure Engine

↓

p_{fill} , slippage, impact, execution alpha

↓

Execution Optimization

↓

Order

AC.56. Dan sekarang kita memiliki prinsip yang sangat penting

Sistem kita tidak lagi bertanya:

"Apakah harga akan naik?"

Melainkan secara bertingkat:

Level 1 — Forecast

$E[R_{t+1} | F_t]?$

Level 2 — Portfolio

Asset combination terbaik?

Level 3 — Execution

Bagaimana cara mendapatkan exposure tersebut?

Level 4 — Risk

Berapa banyak yang boleh kita pertaruhkan?

Level 5 — Management

Apakah posisi masih optimal?

Ini adalah struktur yang jauh lebih dekat dengan **systematic trading architecture** daripada sekadar model prediksi harga.

AC.57. Persamaan inti Tahap AC

Kalau Tahap AB mempunyai:

$$(\hat{\mu}_t, \hat{\Sigma}_t)$$

sebagai output utama, maka Tahap AC dapat diringkas oleh:

$$\alpha_{i,t}^{exec}(a) = P(F | F_t, a) [E(R_i | F, F_t, a) - C_i(a)]$$

dengan action:

$$a = (side, price, quantity, type, TTL).$$

Kemudian:

$$a^* = \arg \max_a \alpha_{i,t}^{exec}(a).$$

Dan portfolio-level system menggunakan:

$$\alpha_t^{exec}$$

sebagai pengganti μ_t mentah.

AC.58. Posisi AC dalam keseluruhan proyek

Sekarang arsitektur kita sudah menjadi:

A-AB

|

|— Market Data

|— Returns

|— Factors

|— Regime

|— Expected Return

|— Covariance

|

▼

AC

|

|— Order Book

|— Spread

|— Depth

|— OBI

|— OFI

|— Trade Flow

|— Queue Position

|— Fill Probability

|— Partial Fill

|— Slippage

|— Market Impact

|— Execution Alpha

|

▼

Portfolio / Execution Optimization

|

▼

QUBO / Ising

|

|— Exact Solver

|— Classical Optimizer

|— VQE

|— QAOA

|

▼

Order

|

▼

Position Management

|

- └ TP
- └ SL
- └ BEP
- └ Trailing
- └ Re-optimization

AC.59. Satu keputusan desain yang menurut saya sangat penting

Jangan langsung memasukkan order book ke QUBO.

Lebih baik kita mempertahankan hierarki:

Microstructure → Execution Parameters → QUBO.

Dengan demikian QUBO tidak menjadi model yang mencoba memahami seluruh market microstructure.

QUBO hanya menyelesaikan problem kombinatorial:

dari banyak kemungkinan keputusan, mana konfigurasi yang optimal?

Ini membuat eksperimen quantum kita jauh lebih terkontrol.

AC.60. Dan ada satu hal yang menjadi sangat menarik

Setelah AC, kita sebenarnya telah memiliki **dua problem optimasi yang berbeda**:

Problem A: Portfolio Selection

$$\min_x [-\alpha^\top x + \lambda x^\top \Sigma x + C(x)]$$

dan:

Problem B: Execution Selection

$$\min_y [-EV(y) + C(y)].$$

Keduanya dapat berupa QUBO.

Artinya, pada akhirnya kita bahkan dapat membangun arsitektur:

Tetapi **jangan digabung dahulu**. Kita harus membuktikan masing-masing layer secara terpisah agar ketika performa sistem berubah, kita tahu penyebabnya.

Dan menurut urutan desain yang telah kita bangun, **Tahap AD** paling tepat digunakan untuk membangun **Trade Opportunity & Entry/Exit Path Engine**: mengubah μ , volatility, regime, dan probabilitas fill menjadi distribusi kemungkinan perjalanan harga sampai **TP/SL**, sehingga kita dapat menurunkan secara matematis **probabilitas TP sebelum SL, expected R-multiple, optimal entry, optimal SL, dan optimal TP**. Ini akan langsung menghubungkan AC dengan mekanisme posisi yang sejak awal kamu inginkan.